Learning-Augmented Algorithms in Real-World Caching

Final Project for CS395T Computer System Performance Analysis

Greyson Baker (gjb885)

3 May 2026

1 Introduction

Advancements in computer science theory are not always immediately applicable to the real problems they are motivated by. Whether due to simplifications made to the problem in the pursuit of the solution or barriers made between the solution method and implementable tools, or otherwise, friction exists between these stages of advancement. The current state of learning augmented algorithms largely mirrors this nature. More recently, the development of learning-augmentation in theory approaches for online problems has yielded strong mathematical results. However, we see far less discussion of their use in real systems with distinct reference to these algorithms.

Caching remains a valuable systems problem to address. Despite the ubiquity of implementations of caching algorithms, human-designed advances continue to have relatively large gaps with optimal solutions. Moreover, due to the near-ubiquity of caching as a tool in systems at a variety of scales, improvement in caching could affect a huge number of devices.

With uses such as, comparatively slower-acting, content delivery networks, using some compute to improve caching results is possible [Torabi et al., 2024]. We consider today the merit that exist at present between possible mathematical algorithmic approaches in their implementations in real-world caching and develop an outline of issues which a good real-world system should consider.

2 Related Works

Some exploration into the empirical analysis of learning-augmented algorithms has been completed in other fields [Silwal, 2021]. Most closely, in Chłędowski et al. [2021], the authors also discuss a variety of learning-augmented algorithms in an empirical analysis with caching. Not only does it, however, use a much heavier weight and offline-learned algorithm [Liu et al., 2020], which our analysis replaces with an online, and with appropriate tuning, higher throughput model. The prior analysis also uses a generic dataset of cache traces, rather than focusing on the most plausible implementation in web-served database access caching.

3 Background

3.1 Machine Learning in Caching

Online nonlinear machine learning approaches show promise at identifying complex relationships between data and their future usage in a stream of work [Song et al., 2020, Liu et al., 2020, Li et al., 2022]. Amongst barriers to their real implementation we identify two: 1) Even simple machine learning algorithms are often more computationally costly than 'simpler' heuristic approaches like FIFO and LRU. For sufficiently compute-available tasks, like the aforementioned content delivery networks, the burden of recalling data can be sufficient that machine learning algorithms could be reasonable. 2) Machine learning algorithms offer few guarantees. Whereas analyses for LRU and FIFO provide reasonable results [Chrobak and Noga, 1999] and a practically solid results from a history of use, this approach offers far more limited guarantees and a smaller scale of employment.

Yet, these algorithms do exist and serve as a valuable tool for considering the implementations of algorithms which, as we will discuss, might leverage them as approximate predictors for complex or hard tasks. For this investigation, we primarily rely upon LRB, a machine learning algorithm for caching which seeks to predict the behaviour of Belady's algorithm, the caching optimal, in an online setting using a history

of recent cache operations [Song et al., 2020].

3.2 Learning-Augmented Algorithms

Lykouris and Vassilvitskii [2020] is often credited with 'kickstarting' the investigation into learning-augmented algorithms and did so in the scope of caching. In this scheme, at least one predictor is used to help solve a portion of the algorithm problem. This predictor is wrapped in some scheme which tries to leverage the information, but also avoid particularly bad outcomes with a bad predictor. Analysis largely revolves around the extremes of a perfect predictor, wherein a k-consistent algorithm achieves a competitive ratio of k, and the worst case predictor, wherein a j-robust algorithm achieves at worst a competitive ratio of j. Usually, results are also parameterized in terms of the error rate of the predictor against the optimal, though the exact definition of that error can vary.

We use three of these algorithms in our investigation; a brief overview of the approaches is merited, due to their impact on the outcome, which proceeds below. It ought be noted that this is not a complete list of existing or even merely notable learning-augmented algorithms in caching. Nor are robustness and consistency the only metrics which might be used in analysis [Sadek and Elias, 2024], however, due to their presence in the original paper, they are some of the only metrics in the space which are used reliably across analyses.

3.2.1 Predictive Marker

The *Predictive Marker* algorithm comes from Lykouris and Vassilvitskii [2020] and achieves a 2-consistency and $O(\log k)$-robustness with the inclusion of random tiebreaks.

We can describe this algorithm as a marking algorithm which uses a predictor to select between unmarked options according to an estimation of next-use. It forms phases by marking incoming elements until all are marked and only allows eviction when both an element is unmarked and, if previously evicted in this phase, was not related to a bad eviction 'too often' as measured by a chain of related evictions. In that case, it instead chooses uniform randomly between unmarked options.

3.2.2 Blind

The algorithm proposed in Wei [2020], which we will term *Blind* in reference to its BlindOracle component and its relative blindness to its robust algorithm, is fundamentally a "Follow the Leader" algorithm. Following its analysis, it achieves a $1+\varepsilon$-consistency and approximately $O(\log k)$-robustness.

This algorithm leverages two algorithms, one a predictor and one which we consider robust. Both algorithms are allowed to maintain their own shadow cache which tracks their behavior on the input sequence without regard to each other or the real cache. On every step, the algorithm which has, so far, achieved fewer cache misses guides the following step as the leader. The algorithm then uniform-randomly selects from items not in the leaders cache to select an eviction.

3.2.3 Guard

Lastly, we consider an improvement on Lykouris and Vassilvitskii [2020], Chen et al. [2025]. The *Guard* algorithm is another marking type algorithm, though with a conceptual approach somewhat closer to using a failure budget. This algorithm achieves 1-consistency and a $O(\log k)$-robustness.

The *Guard* algorithm uses a phase which tracks all unrequested old pages of a cache and continues until all of those old pages are requested. At any step, if the incoming page had been previously evicted in the current phase, it is guarded from future eviction in the phase and a uniform-random unguarded page is evicted. Otherwise, the predictor is blindly followed on selections between unguarded pages.

3.3 Caching Simulation

The basis of the caching simulator we utilize is *libCacheSim*, which enables testing algorithms in a consistent environment against traces across cache sizes [Yang et al., 2020, 2023a,b]. Moreover, it keeps some track of relative throughput and easy use of content server related traces.

4 Methodology

First, we seek to determine an empirical result for the performance of the algorithms given a particular performance by the machine learning algorithm. Although bounds already exist, the opportunity to in-

vestigate where the results lay is independently valuable. Second, we seek to compare the relative results of the algorithms in different cases, identifying their individual strengths and weaknesses in various cache size. Third, we consider that analysis again, but for cache types. Fourth, we seek to consider runtime affects for the different methods of the base predictor and the algorithm wrappers.

As an aside, note the investigation's lack of two testing paradigms. We do not seek to discuss behavior regarding exceedingly bad predictor behavior. The reason for this is rather simple, any exceedingly bad predictor is unproductive to be used in practice and the mathematical analyses of robustness already provide worst case bounds across both adversarial outputs and adversarial predictor results. We also do not seek to discuss perfect predictor behavior in this investigation. Although there is some discussion to be had regarding decay of quality on real traces for a perfect oracle then given error, we will largely leave that for mathematical discussion, as was done in Lykouris and Vassilvitskii [2020]. That discussion does not meaningfully consider the actual nature of learned predictors and, although perhaps independently useful, is not the focus of this investigation.

The base predictor for our test, *LRB*, learns online over a window size and is leveraged to predict across a group which page is best to evict. In order to approximate some of the range between our example predictor and a "worse but plausible" predictor, a noise term is used. In a predictor instance with noise p, there is a p probability that the predictor swaps to a noised mode, wherein it draws between legal options at uniform random. For a variety of small and at-scale cache sizes, each of the learning-augmented algorithms will be employed and compared against each other, the base predictor used blindly, and default implementations of *FIFO* and *LRU*. Optimal *Belady's* will also be considered for reference. Our key metric is miss rate, with this being the only appropriate metric given a simplification made.

These tests are conducted across a variety of traces. To To improve testing efficiency, two huge simplifications are employed. For fairness between algorithm designs and to better fit their use case, we simplify objects to be of equal size. For runtime efficiency, we limit the traces to five million steps and perform a number of tests on smaller cache sizes. Also for fairness, we keep to the expectation of the algorithms and perform a ranking and selection decision across all legal options. Specifically to address the fourth expectation, we also run some sample tests with comparison between runtime results for the approaches and miss rate results.

In total, six traces across three categories are used, summarized in Table 1 [Berg et al., 2020, Yang et al., 2020, Song et al., 2020, Li et al., 2020]. For reference, a higher Zipf slope suggests a higher concentration of requests to a smaller group of objects than a trace with a lower Zipf slope.

5 Analysis

LRB, the base predictor, has meaningful variability in performance relative to optimal across different cache types. Although on some caches it does very well, sometimes the noising strategy yields better results. That is, a uniform random selection performs better in terms of cache misses than the predictor does. Critically, it does sometimes perform meaningfully better than *LRU* and *FIFO*, so there is clear merit to some usage. In Figure 1, the cache miss rate has been modified so $s = \frac{c_A - c^*}{1 - c^*}$ where c is the miss rate for either the algorithm c_A or the optimal c^*.

Blind follows the leader very closely. On all tests, *Blind* sticks tightly to the better of *LRU*, which serves as the robust algorithm used, and the predictor, with the most extreme improvement on the 64KB version of Meta KV wherein it achieves a miss rate .0035 lower (or .15%). . It does in some configurations perform worse. Indeed, 8% of the configurations tested had a result more than .15% worse than the minimum of the hit rates for *LRU* and the noised *LRB*, though the maximum of that gap was 1.15% worse miss rate.

PredictiveMarker does see success beyond the min of *LRU* and *LRB*, yet it also sees failure to that min which is relatively large. Exemplified visually in Figure 2, *PredictiveMarker* does achieve successes as large as 7.6% over *LRU*, in Alibaba Block at a 2KB cache size with no noise. However, that performance does worse than blindly following *LRB* by 12.1%. Meanwhile, it also sees a failure of 3.8% against *LRU* on Alibaba Block at .5KB Cache size with a uniform random predictor. Generally, *PredictiveMarker* is characterizable as staying close to *LRU* throughout all traces.

Guard tends to achieve better-than-predictor results on both easy and hard problems for predictors. We'll define an easy problem for the predictor in this con-

Table 1: Summary statistics for the evaluated access traces

Trace	# Objects	Top Object (%)	Zipf Slope	Duration (days)
Meta KV	703,195	5.30	0.9257	0.015
Twitter KV	29,644	7.23	1.1079	0.128
Wikipedia Object	1,290,768	1.29	0.7507	0.013
Meta Object	2,461,926	1.42	0.5004	0.465
Alibaba Block	430,243	0.40	1.1714	7.860
Meta Block	2,487,687	0.02	0.5805	0.375

Figure 1:

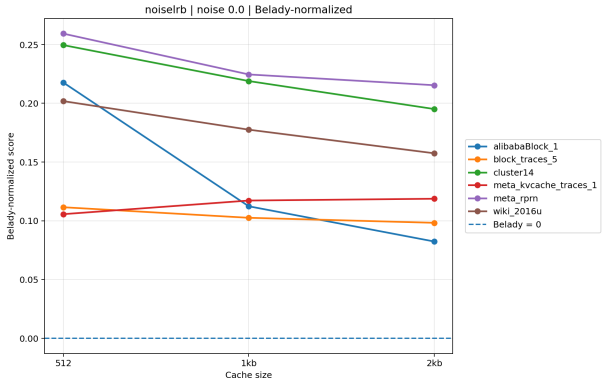
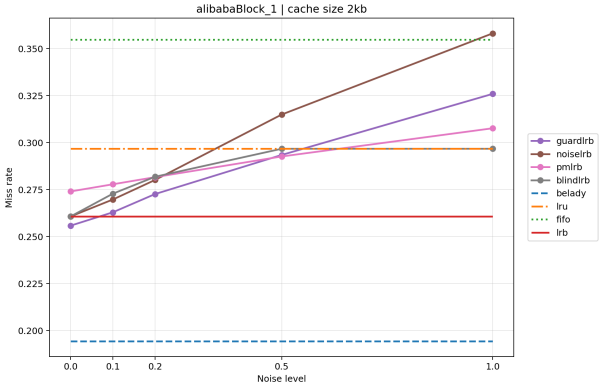


Figure 2:



text as one where the predictor achieves good results relative to *LRU* and in which those results decay as noise is added to the predictor. Hard problems will be those with either performance meaningfully worse than *LRU* or *FIFO* and those where performance does not vary widely, or improves, as noise is added. Over 10% of problems saw an increase against the predictor of over 3% using noiseless *LRB*. Even moreso against *LRU*, on average. Amongst easy problems like Alibaba Block's 2KB instance, *Guard* still achieves a miss rate 1.9% better than the predictor as seen in Figure 2. We identify Twitter Block's 2KB variant as a distinctly hard problem for the predictor, considering that both the outlined conditions were met, yet it still achieves a 10.3% improvement over *LRU* and a 16.4% improvement over *LRB* at noiseless.

Yet, at high noise, not only does *Guard* perform worse than *LRU* in every trace attempted, it also performs worse than the other algorithms in 89% of tested traces at a noise of 1.

One of the basic expectations in good caching algorithms is that larger caches yield better results. All of the caching algorithms explored tend to obey this expectation in terms of hit rate. Moreover, the learning-augmented caching algorithms improve faster than the optimal, but slower than *LRU* and *FIFO*. The LSQR linear slope of this improvement in absolute score is summarized in Table 2. As an aside, some datasets, namely Meta KV in this experiment, do get worse results using *LRB* as cache size increases from .5KB to 2KB, yet improve again when approaching 64KB. Although the exact reason for this isn't clear, that the phenomenon exists is concerning.

However, ranking all legal options as done in this investigation is a burden that increases with cache size. These algorithms are already much slower than *FIFO* and *LRU*: the simulator for *LRB* achieves roughly a tenth of the throughput. This is despite the model training and inference leveraging twelve times the compute as normal simulator execution. One solution proposed but not mandated innately in *libCacheSim* is to randomly sample a comparatively small group of candidates from which to rank and make decisions. The results are shown in Table 3.

Table 2: Cache Miss Slope of Algorithms Across Cache Sizes

Algorithm	**Cache Miss Median Slope**
Optimal	-0.015
FIFO	-0.026
LRU	-0.022
LRB	-0.019
PredictiveMarker	-0.021
Blind	-0.020
Guard	-0.018

Table 3: Miss Rates at 2KB Cache Size for Full and Sampled Variants

Trace	**PM**		**Blind**		**Pure LRB**	
	Full	**Samp.**	**Full**	**Samp.**	**Full**	**Samp.**
Meta KV	.4004	**.3781**	.3917	**.3777**	.3928	**.3781**
Wiki Object	**.7306**	.7366	**.6617**	.6631	**.6617**	.6629
Alibaba Block	**.2661**	.2740	.2607	**.2552**	.2607	**.2539**

6 Discussion

Broadly, *LRB* is not an unsuccessful tool in its own right, but the data suggests that not only can it easily be brought to have minimum guarantees (with any of the algorithms), it can also exceed the score of the predictor often (with *Guard*) or be guaranteed to achieve hit-rate performance not-worse than an alternative (with *Blind*). On a variety of cache sizes, trace types, and for lower error levels, *Guard* in particular achieves good results. However, the injection of additional heuristics and increasing the budget for failure, although likely the fundamental origin of being able to achieve good success, can yield worse results in particular cases. Yet, this injection also enables better-than-predictor performance for portions of the workload that are evidently more difficult for the predictor to react to.

The fourth considerations requires a more nuanced deliberation. Even with a fairly lightened model, training and running *LRB* is slower than most heuristic approaches, notably *LRU*. The data on sampling vs full ranking suggests that this gap can be shrunk massively, but having algorithms that scale against cache size in runtime and precision is not conducive to the general desire to leverage larger caches as more expensive, but more powerful, tools to caching problems.

The success of *Guard* even in comparison to the others suggests that the prior schemes for construction were fundamentally misdirected. Achieving good consistency, robustness, and other metrics certainly have their values. The practical difference often lies in the middle however, being able to achieve better-than-predictor results with algorithm construction and finding analytical tools to determine when and why this occurs from a theory perspective would likely open the door to rapid advancement in *practically* useful learning-augmented algorithms.

Regarding using sampling instead of full ranking, although the results are slightly worse occasionally, Table 3 displays that the detriment to sampling is not large for this dataset. A key limitation is that this does not address training time compute consumption. Plausibly training could be made less frequent to accommodate as necessary, but this involves its own secondary tradeoff in regards to adaptation speed in the online setting. Indeed, the major burden with learning-augmented caching may be finding a mix of protections strong enough and predictors light enough to do slightly better than existing algorithms without large compute burdens. Seeking 1-consistency, in that framework, is neither a sufficient description (as discussed above) nor perhaps a necessary one, as a lightweight and ultra-high quality predictor may be too contradictory to exist economically.

References

Benjamin Berg, Daniel S Berger, Sara McAllister, Isaac Grosof, Sathya Gunasekar, Jimmy Lu, Michael Uhlar, Jim Carrig, Nathan Beckmann, Mor Harchol-Balter, et al. The {CacheLib} caching engine: Design and experiences at scale. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 753–768, 2020. URL https://www.usenix.org/system/files/osdi20-berg.pdf.

Peng Chen, Hailiang Zhao, Jiaji Zhang, Xueyan Tang, Yixuan Wang, and Shuiguang Deng. Robustifying learning-augmented caching efficiently without compromising 1-consistency, 2025. URL https://arxiv.org/abs/2507.16242.

Marek Chrobak and John Noga. Lru is better than fifo. *Algorithmica*, 23(2):180–185, 1999. URL https://link.springer.com/content/pdf/10.1007/PL00009255.pdf.

Jakub Chłędowski, Adam Polak, Bartosz Szabucki, and Konrad Zolna. Robust learning-augmented caching: An experimental study, 2021. URL https://arxiv.org/abs/2106.14693.

Chunhua Li, Man Wu, Yuhan Liu, Ke Zhou, Ji Zhang, and Yunqing Sun. Ss-lru: a smart segmented lru caching. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*, pages 397–402, 2022. URL https://dl.acm.org/doi/pdf/10.1145/3489517.3530469.

Jinhong Li, Qiuping Wang, Patrick P. C. Lee, and Chao Shi. An in-depth analysis of cloud block storage workloads in large-scale production. In *2020 IEEE International Symposium on Workload Characterization (IISWC)*, pages 37–47, 2020. doi: 10.1109/IISWC50251.2020.00013.

Evan Zheran Liu, Milad Hashemi, Kevin Swersky, Parthasarathy Ranganathan, and Junwhan Ahn. An imitation learning approach for cache replacement, 2020. URL https://arxiv.org/abs/2006.16239.

Thodoris Lykouris and Sergei Vassilvitskii. Competitive caching with machine learned advice, 2020. URL https://arxiv.org/abs/1802.05399.

Karim Ahmed Abdel Sadek and Marek Elias. Algorithms for caching and MTS with reduced number of predictions. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=QuIiLSkt04.

Sandeep Silwal. *Learning-augmented algorithms*. PhD thesis, Massachusetts Institute of Technology, 2021. URL https://dspace.mit.edu/handle/1721.1/139333.

Zhenyu Song, Daniel S. Berger, and Kai Li an dWyatt Lloyd. Learning relaxed belady for content distribution network caching. In *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation*, 2020. URL https://www.usenix.org/system/files/nsdi20-paper-song.pdf.

Hoda Torabi, Hamzeh Khazaei, and Marin Litoiu. A learning-based caching mechanism for edge content delivery. In *Proceedings of the 15th ACM/SPEC International Conference on Performance Engineering*, ICPE '24, page 236–246. ACM, May 2024. doi: 10.1145/3629526.3645037. URL http://dx.doi.org/10.1145/3629526.3645037.

Alexander Wei. Better and simpler learning-augmented online caching, 2020. URL https://arxiv.org/abs/2005.13716.

Juncheng Yang, Yao Yue, and K. V. Rashmi. A large-scale analysis of hundreds of in-memory cache clusters at twitter. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 191–208. USENIX Association, 2020. ISBN 978-1-939133-19-9. URL https://www.usenix.org/conference/osdi20/presentation/yang.

Juncheng Yang, Ziyue Qiu, Yazhuo Zhang, Yao Yue, and K.V. Rashmi. Fifo can be better than lru: The power of lazy promotion and quick demotion. In *Proceedings of the 19th Workshop on Hot Topics in Operating Systems (HotOS23)*, page 70–79. Association for Computing Machinery, 2023a. ISBN 9798400701955. doi: 10.1145/3593856.3595887.

Juncheng Yang, Yazhuo Zhang, Ziyue Qiu, Yao Yue, and K.V. Rashmi. Fifo queues are all you need for cache eviction. In *Symposium on Operating Systems Principles (SOSP'23)*, page 130–149. Association for Computing Machinery, 2023b. ISBN 9798400702297.